

Advances in Microprocessor Cache Architectures Over the Last 25 Years

Ravi Iyer , Vivek De , Ramesh Illikkal, David Koufaty , Bhushan Chitlur, Andrew Herdrich, Muhammad Khellah, Fatih Hamzaoglu, and Eric Karl, Intel Corporation, Hillsboro, OR, 97124, USA

Over the last 25 years, the use of caches has advanced significantly in mainstream microprocessors to address the memory wall challenge. As we transformed microprocessors from single-core to multicore to manycore, innovations in the architecture, design, and management of on-die cache hierarchy were critical to enabling scaling in performance and efficiency. In addition, at the system level, as input/output (I/O) devices (e.g., networking) and accelerators (domain-specific) started to interact with general-purpose cores across shared memory, advancements in caching became important as a way of minimizing data movement and enabling faster communication. In this article, we cover some of the major advancements in cache research and development that have improved the performance and efficiency of microprocessor servers over the last 25 years. We will reflect upon several techniques including shared and distributed last-level caches (including data placement and coherence), cache Quality of Service (addressing interference between workloads), direct cache access (placing I/O data directly into CPU caches), and extending caching to off-die accelerators (CXL.cache). We will also outline potential future directions for cache research and development over the next 25 years.

CACHING CHALLENGES OVER THE LAST 25 YEARS

 ver the last 25 years, we have seen significant advances in microprocessors including substantial improvements in core frequency and performance, multicore, manycore, and more recently heterogeneous compute architectures [diverse central processing unit (CPU) cores and tightly coupled accelerators/devices]. These have enabled applications to grow rapidly from single-threaded to multithreaded on client and server platforms and furthermore to multitenant and service-oriented microservices scenarios in virtualized cloud infrastructure. All these advancements in compute performance and increase in application demands required improvements in access to data (both latency and bandwidth). With a slower pace of advancements in dynamic random access memory

(DRAM), architects had to innovate and advance caching techniques to facilitate high bandwidth, low latency data access from the core as well as from input/output (I/O) devices and accelerators. Figure 1 illustrates the compute growth and the memory wall challenge and highlights some caching innovations that we will cover in the article. The advancements in caching (as a result) are best described by an illustrative example (see Table 1). About 25 years ago, the Intel Pentium Pro was launched into the market for both client and eventually server platforms. The Intel Pentium Pro was a single-core processor running at 150–200 MHz and featured an off-die but on-package nonblocking L2 cache (256 K at introduction) connected to the core using a backside bus to address the memory latency and enable concurrent access to cache and memory. Fast forward 25 years to our current generation server microprocessors, which have tens of cores (each multithreaded and capable of running at well over 3 GHz) with on-die cache capacity at almost 100 MB or more (including both L2 and L3) that is physically distributed across an on-die interconnect. Table 1 presents the number of cores, frequency, and cache sizes for the latest third-generation

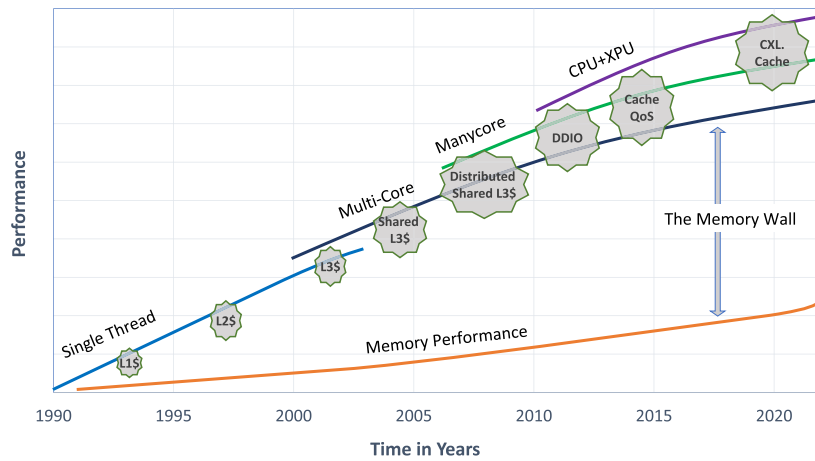


FIGURE 1. Growing compute versus memory gap necessitates advances in on-die cache hierarchy and novel features.

Intel Xeon scalable server processor (formerly Icelake-SP)¹ in comparison with Intel Pentium Pro.

IN THIS ARTICLE, WE WILL DESCRIBE THE LAST 25 YEARS OF ON-DIE CACHING ADVANCEMENTS BY DISCUSSING THE CHALLENGES THAT THEY ADDRESSED AS WELL AS THE INNOVATIONS THAT ENABLED DATA ACCESS AND DATA MOVEMENT EFFICIENCY WITHIN THE MICROPROCESSOR AS WELL AS WITH TIGHTLY COUPLED DEVICES AND ACCELERATORS.

In this article, we will describe the last 25 years of on-die caching advancements by discussing the challenges that they addressed as well as the innovations that enabled data access and data movement efficiency within the microprocessor as well as with tightly coupled devices and accelerators. It should be noted that excellent advances in caching occurred in other areas of the platform architecture as well (e.g., use of DRAM as a cache with the emergence of Intel Optane persistent memory), but we limit our discussion in this article to on-die static random access memory (SRAM) caches in the microprocessor and later in devices/accelerators on server platforms. Figure 2 illustrates the on-die caching challenges and advancements that we will cover in more detail in subsequent sections. The first of these challenges was at the advent of multicore processors, where the growing last-level cache (initially L2 and later

L3) could remain private or be shared across subset or all cores. The second challenge emerged as the number of cores on-die was scaled and this made it difficult to have a single monolithic shared last-level cache (L3) due to the latency increase and physical placement considerations. Furthermore, as virtualization and cloud computing emerged, there was also a question of how resources should be shared as well as performance isolated across multiple applications or tenant virtual machines (VMs) running simultaneously on the microprocessor, sharing the L3 cache in the cloud infrastructure. These challenges necessitated the physical distribution of L3 slices across cores and introduced additional questions such as 1) how should data be placed across the L3 slices and across the hierarchy? and 2) should the caches enable some partitioning to

TABLE 1. Example of core/caching advances in 25 years.

	Intel Pentium Pro (1996)	Third Gen Intel Xeon Scalable (2021)
Cores (C) and Threads (T)	1C, 1T on 0.35 μm	Up to 40C, 2T per core on 10 nm
Core Frequency	150–200 MHz	Up to 3.9 GHz base frequency
L1 Cache (code, data)	8 KB Code 8 KB Data	32 KB Code 48 KB Data
L2 Cache (unified)	256 KB to 1 MB	1.25 MB per core on-die
L3 Cache	None	Up to 60 MB shared by cores; on-die distributed layout w/1.5 MB per core

Intel, the Intel logo and Xeon are trademarks of Intel Corporation or its subsidiaries.

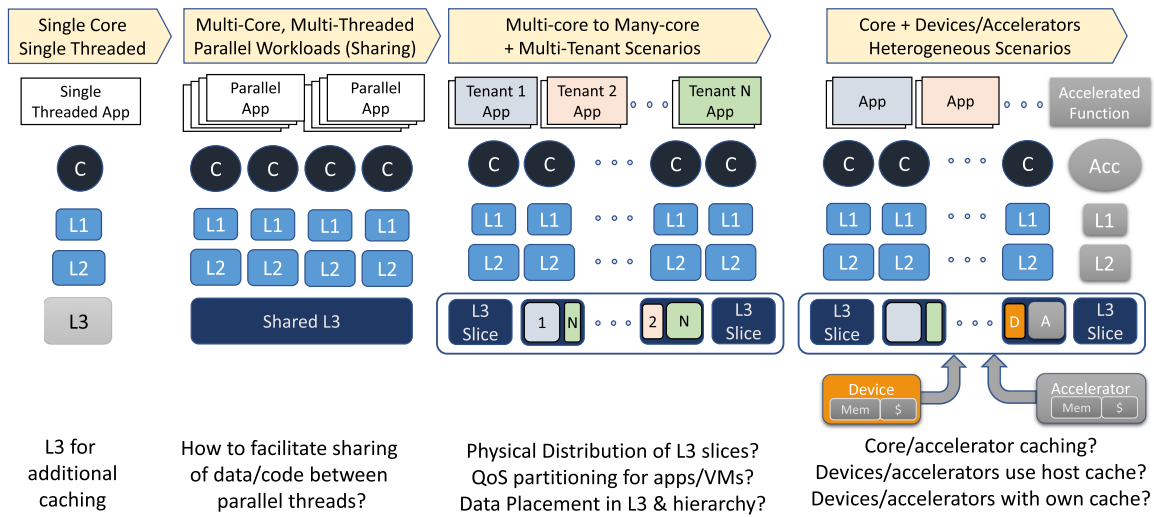


FIGURE 2. Caching challenges and advances over 25 years.

facilitate resource monitoring and allocation for quality of service? Last but not least, as I/O devices (e.g., networking and storage) started playing a critical role in cloud deployments, improving the data movement between the device and the host platform processing the content became increasingly important to enable faster network speeds. Moreover, with the emergence of domain-specific accelerators [primarily off-die but also integrated graphics processing units (GPUs) on-die], the placement and sharing of data across the CPU, devices and accelerators needed to be considered carefully.

In this article, we will delve into the architectural solutions that have emerged and been integrated into microprocessors over the years to address the above questions. We will also discuss the design and technology advancements that improved cache density, reliability, yield, and latency. These foundational improvements ensured that we continue to scale and grow our cache subsystems in microprocessors keeping up with the ever-increasing application demands.

SHARED CACHES IN MULTI/MANY-CORE CPUs

As commercial multicore x86 CPUs started emerging about 15 years ago, the traditional approach was to just have individual cache hierarchies (L1, L2) on each core. However, there was an opportunity to further improve sharing and communication between the cores with a shared last-level cache. Initially, as dual-core CPUs started emerging the focus was on a shared L2 cache. But soon, the use of a shared L3 as last-level cache started to become more common with quad core and larger multicore CPUs in order to share code as well as

data amongst parallel applications. The shared L3 in quad core CPUs was initially designed as a monolithic cache and as the core count in the die increased, the size and latency of the L3 became a challenging architecture/design tradeoff. With more cores on die, more sophisticated on-die interconnects (rings, mesh) started to be considered for scalability and intercore communication. This also led to the consideration for architecting the shared L3 as multiple distributed cache slices across the interconnect, with each slice potentially collocated with a core but accessible by all cores.

With a distributed shared L3, while it was easier to distribute the cache space and reduce latency on a per cache slice, there were other considerations that became important: 1) how do we place data across the slices? 2) should the L3 cache be inclusive? and 3) how do we maintain coherence with respect to the lower (L1/L2) levels of cache that are on each core? To avoid hotspots in the interconnect, the placement of data was based on hashing techniques that distributed the data evenly across the L3 slices. Since the L1/L2 caches were small (<256 KB) initially, the large L3 slices (2–3 MB) could be kept inclusive and thereby keep the protocol complexity low. To maintain coherence, the L3 cache slices started supporting valid bits to keep track of lines in lower levels of cache.

As additional cores continued to be integrated and the interconnect became larger (ring to mesh, for example), the latency to the last-level caches started to increase as well. There were three considerations that emerged with the growing core count and interconnect on-die: 1) the number of integrated memory controllers on-die started to increase to enable higher bandwidth;

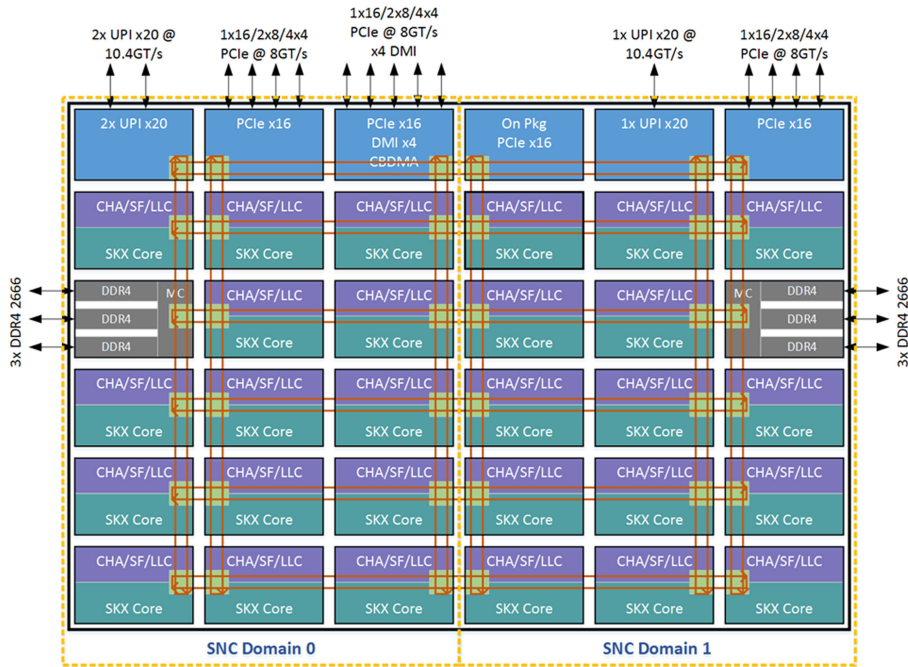


FIGURE 3. Shared L3 in Intel Xeon Scalable Processor.¹⁶

2) the bits needed in the last-level cache to track coherence state (of each cache line in the lower level caches) started to increase; and 3) whether to grow the L2 cache, scale the L3 cache, or optimize the data placement to reduce latency. In order to enable flexibility in the cache hierarchy and data placement for locality, enhancing the L3 architecture and policies was desirable. As a result, techniques such as noninclusive caches but inclusive directories (NCID)² emerged to provide flexibility and efficiency. In the first-generation Intel Xeon scalable server processors (previously known as Skylake SP) with up to 28 cores per socket, the interconnect became a scalable mesh, the L2 cache per core grew in size (from 256 KB to 1 MB), and the L3 cache changed from inclusive to noninclusive to enable flexible placement of data in L2 and L3. Although a line in L2 did not require a data copy in the (noninclusive) L3, the L3 cache slice supported (inclusive) snoop filters to maintain coherence tracking the lines in lower levels of the cache.

Last but not least, as the number of memory controllers and the number of L3 cache slices grew on die, the placement of data for locality was also a key consideration. In Skylake, sub-nonuniform memory access (NUMA) clustering was introduced to enable multiple domains within a die, where each domain consisted of a subset of cores, L3 slices, and integrated memory controllers. The sub-NUMA clustering approach enabled the placement of data coming from one memory

controller into L3 slices that are closer to it, whereas placing data from another memory controller into L3 slices closer to that. This reduced the latency of data access using NUMA affinity extended into the on-die architecture while still ensuring no replication of data blocks occur with the L3 slices.

Figure 3 shows how the first-generation Intel Xeon Scalable Family server microprocessor (Skylake SP)³ enables all of the above capabilities in the on-die last-level (L3) cache. In addition to the hardware capabilities, it soon also became important to enable software hints to modulate placement in different levels in the cache hierarchy. For example, prefetch instructions to pull data closer in the cache hierarchy, and instructions such *CLDEMOTE*¹⁷ to push data further away from the originating core so that it is accessible by another core who may find it at the right level in the cache hierarchy, reducing coherence penalties. Beyond such software control, additional software guidance of cache allocation and monitoring of cache usage is discussed below to enable QoS capabilities that address shared resource contention.

QoS IN SHARED CACHES

As chip multiprocessors grew in the number of cores and deployments became popular in cloud environments, the number of workloads running simultaneously on these processors also grew rapidly. This was fueled by virtualization and multitenant hosting

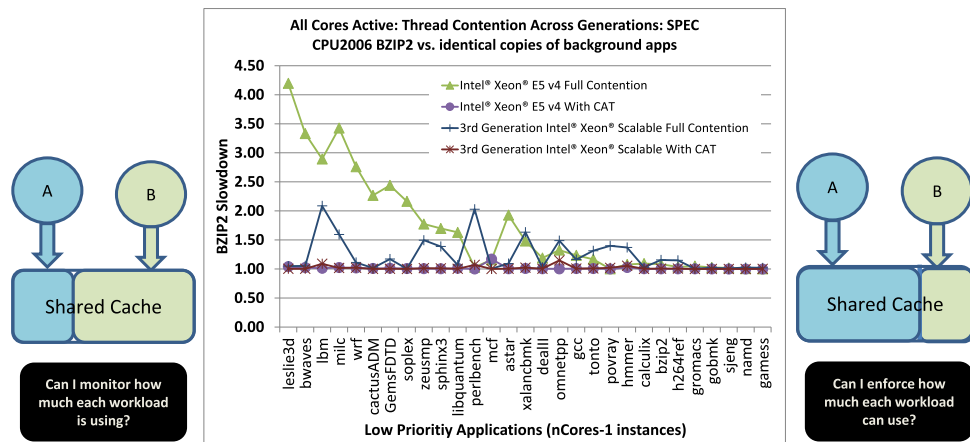


FIGURE 4. Cache QoS: Addressing shared cache contention.

(many VMs) and has since evolved into the use of containers and microservices as well. With many simultaneous and disparate workloads running on the cores, contention for shared resources such as last-level cache led to performance variability, long tail latencies to complete certain operations, or steady-state performance imbalance. This QoS problem⁴ was highlighted initially over 15 years ago and led to innovative research in managing shared resource contention.⁵

An example of the shared resource contention effects is shown in Figure 4 where a high priority workload (BZIP2 running simultaneously with many other low priority workloads) can slow down by as much as 4X due to shared cache contention in certain microarchitectures. To address these concerns in mainstream server microprocessors, the first QoS techniques in hardware were integrated into Intel Xeon E5/E7 v3 and v4 processors⁶ enabling last-level cache monitoring (CMT), memory bandwidth monitoring (MBM), and cache allocation controls (CAT). With CMT and MBM, execution environments could dynamically monitor the LLC capacity and memory bandwidth consumed by software threads using resource monitoring IDs (RMIDs). With CAT, execution environments could allocate different amounts of last-level cache space using different classes of

service (CLOS) and associated bit vectors to represent cache space needs. Subsequent generations expanded these controls to include memory bandwidth allocation. These QoS capabilities and architecture made possible by the Intel Resource Director Technology (RDT) enabled dynamic monitoring and control of contention effects, independent scaling of architectural parameters (such as RMIDs and CLOS) and provide a scalable framework.

Resource contention impact can vary considerably depending on other simultaneously running workloads and core/cache hierarchy configurations. As shown in Figure 4 (for BZIP2), cache/memory resource contention on Intel Xeon E5 v4 may result in a performance slowdown of up to ~4X, whereas the same workload on today's third-generation Intel Xeon Scalable processors suffers up to ~2X slowdown. Employing QoS in cache (CAT) using Intel RDT available in these platforms enables prioritization and isolation of high priority workloads while running simultaneously with low priority workloads. As shown in Figure 4, the effect of contention from low priority workloads on the performance of a high priority workload (BZIP2) can be reduced from over 2X (without CAT) down to negligible slowdowns (with CAT) by prioritizing cache space for the high priority application and limiting resource allocation for low

TABLE 2. Cache QoS: Example reduction in slowdown, average to max (SPEC CPU2006 BZIP2 versus background apps).

	Intel Xeon E5 v4 Full Contention	Intel Xeon E5 v4 With CAT	Third Gen Intel Xeon Scalable Full Contention	Third Gen Intel Xeon Scalable With CAT
Average	1.71	1.01	1.24	1.02
Geomean	1.54	1.01	1.21	1.02
Max	4.20	1.17	2.09	1.15
Min	1.00	1.00	1.00	1.00

priority applications. Table 2 presents benefits for the BZIP2 example in more detail indicating that average, geometric, and maximum performance degradation can be addressed using QoS techniques in shared caches.

CACHE ADVANCEMENTS FOR I/O DEVICES AND ACCELERATORS

Traditionally, I/O devices have interfaced with the host using links such as PCIe. Over the years, these I/O devices have increased in data/processing rates (e.g., networking from <100 Mb/s to 1 Gb/s to 10 Gb/s to hundreds of gigabits per second) and new domain-specific processing units have emerged (deep learning, infrastructure processing, etc.) to work closely with host compute platforms. In the former case (increasing data rates for networking), a significant challenge was that incoming packets were initially placed in memory (typical direct memory access operations from the device) and the cores had to read the data from memory for protocol processing and eventually to hand the data off to applications. This overhead significantly limited the amount of network processing that could be done by the host efficiently. In order to address this issue, researchers proposed direct cache access⁷ that allows devices to place data directly into processor caches (e.g., last-level cache) so that the data is closer to the core that processes the packets. The Intel Data Direct I/O (Intel DDIO) feature was introduced in Intel Xeon E5 processors and enables server network interface cards (NICs) to place data directly in processor caches without the detour to memory. This capability improves latency, reduced memory bandwidth and overall packet processing efficiency on the server platform. More recently, in the article by Yuan *et al.*,⁸ carefully managing the amount of cache space allocated to DDIO versus the tenants running on the CPU cores has shown that any performance interference between DDIO placement and core usage can also be managed using RDT/QoS features (CAT as explained in the previous section). In addition to DDIO, new transactions on the coherent fabric (e.g., RdCur) enable I/O devices to read data from the CPU caches without requiring state changes, thereby improving performance by avoiding snoops to IO controllers on subsequent reads/writes from cores.

Next, we turn our attention to more sophisticated accelerator devices like deep learning and infrastructure processing. As the performance requirements and usages have evolved many of these accelerators have incorporated local (device-attached) memory and caches to minimize latency and improve throughput, but still rely on software for bulk data transfers to copy

or flush data at synchronization points. In such scenarios, a high-speed link that provides coherency between host and accelerator has become important to minimize latency, enable new usage models, and simplify the software stack. One of the leading efforts to standardize the interfaces of coherently attached accelerators is driven by the Compute Express Link (CXL) consortium.⁹ CXL implements three logical protocols (CXL.io, CXL.cache, and CXL.mem) on a single physical interface. CXL.io is primarily targeted for streaming bulk data transfer very much like PCIe. CXL.mem enables device-attached memory to be part of OS-managed system memory. CXL.cache is designed for low latency coherent transfer of cache lines between host and device. These three complimentary protocols allow a device to become a cache coherent accelerator with capabilities and performance that match those of its host CPU, with the option of having OS-managed system memory directly connected to the device.

CXL.cache extends the same benefits from caching on the CPU to accelerators, allowing accelerators to exploit spatial and temporal locality, prefetch data, and implement atomic operations locally. Unlike symmetric coherency protocols like UPI or CCIX that require all agents to implement a home agent, CXL implements an asymmetric CXL.cache protocol that relies on the host home agent to maintain coherency between CPU and accelerator. This greatly simplifies the accelerator design since it does not need to orchestrate coherency and instead focuses on providing the core functionality of the device and the simple set of abstracted commands (reads, writebacks, streaming writes, snoops). In addition, the host is responsible for managing coherency for all device-attached memory that is exposed to the host using CXL.mem.

CXL.cache operates exclusively on physical addresses. Devices rely on existing PCIe address translation services to obtain virtual to physical address translations and typically use a device TLB to store the translations. CXL.cache uses 64 B as the coherency granularity and implements a MESI protocol (a cache coherence protocol based on four states: modified, exclusive, shared, and invalid). In addition to using CXL.cache as a low-latency link, coherent attached devices can also improve performance by implementing custom operations like atomics. CXL.cache transactions are used to maintain coherency between the CPU and the accelerator, ensuring that the data remains coherent across components and memory. As is the case with CPU, accelerators use these transactions to obtain a copy of the line, get ownership of the line, etc. However, to enable an accelerator direct access to device-attached memory at high performance, CXL introduced a bias-based

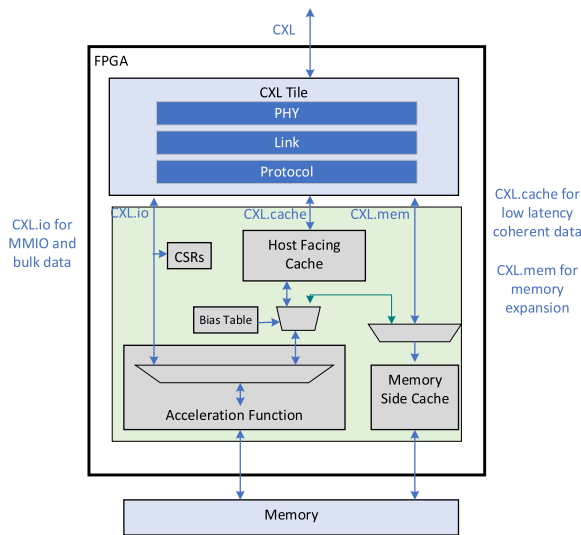


FIGURE 5. CXL-enabled FPGA for acceleration and memory expansion.

coherency mechanism. When the line is in device bias, the host is guaranteed not to have a copy of the line and the accelerator can bypass the host and access the data directly. When the line is in host bias, the accelerator must send a transaction to the host in order to resolve coherency before accessing the data. The bias mode can be controlled by hardware or software and implemented at different granularities.

Intel's CXL-enabled field-programmable gate arrays (FPGAs)^a are a good example of how all of the above CXL technologies can come together to allow the FPGA to act as a coherent accelerator, or a memory expansion bridge, or a combination of both. Intel's Agilix family of FPGA implements CXL as a combination of a hardened tile and soft logic. The tile implements the CXL Phy-Link-Protocol functionality, whereas the soft IP implements the host facing cache subsystem, memory side cache, memory subsystem, bias tables, and the acceleration function logic. Figure 5 illustrates this capability.

CACHE DESIGN CHALLENGES AND EVOLUTION

Design of a hierarchy of cache memory arrays for complex SoCs in scaled CMOS process nodes demands careful optimizations across density, minimum operating voltage (V_{min}), reliability, yield, latency, and process

complexity. The SRAM cells in the L2/L3 cache arrays are the most critical limiters of V_{min} , reliability, and yield [see Figure 6(a)]. As voltage is reduced, bit failure rates increase exponentially. The overall die failure is limited by the worst bits that fail due to device parameter mismatches or defects in the SRAM cell. Thus, the die failure rate goes up with the number of bits. The V_{min} of a large number of dies in high-volume manufacturing is set by the worst dies that meet the target yield. In addition, extra V_{min} margin is added to account for radiation-induced soft errors, erratic bits, supply noises, and aging-induced V_{min} degradation. Therefore, as process variation impacts worsen with technology scaling, the V_{min} becomes extremely difficult to scale.

V_{min} , reliability, yield, and latency can be improved by using bigger transistors in the SRAM cell [see Figure 6(b)], at the expense of array density. Bigger transistors reduce device mismatches and vulnerability to defects, and afford more optimal ratioing of transistor sizes to improve SRAM cell stability. 8T SRAM cells offer a new trajectory of V_{min} versus density tradeoff by eliminating frequency-independent functional read failures. The V_{min} is then limited by writability and data retention. 8T cells are much bigger than 6T cells but offer much better V_{min} . They are used in cache arrays in the lower levels of the hierarchy where V_{min} and read latency are more critical than density.

THE LAST 25 YEARS SAW TREMENDOUS ADVANCES IN CACHING, ESPECIALLY AT THE L3 CACHE, ENABLING DISTRIBUTED BUT SHARED CACHES, QoS AWARENESS, GUIDING I/O DATA PLACEMENT, ENABLING TIGHTER COUPLING WITH ACCELERATOR MEMORY AND CACHING AND MORE.

V_{min} needs to be reduced more aggressively as more cores are added, in order to fit within a fixed maximum power limit. Multivoltage designs, where different voltages are used for the small-signal 6T and large-signal 8T SRAM arrays, improve the overall V_{min} of the SoC [see Figure 6(c)]. Then, V_{min} of individual cache arrays no longer limit the V_{min} of the compute cores that consume most of the power budget. Voltage level shifters, designed to minimize area, power, and delay overheads, are used between the different

^aFor more info, see <https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/solution-sheets/intel-agilix-fpga-product-brief.pdf>

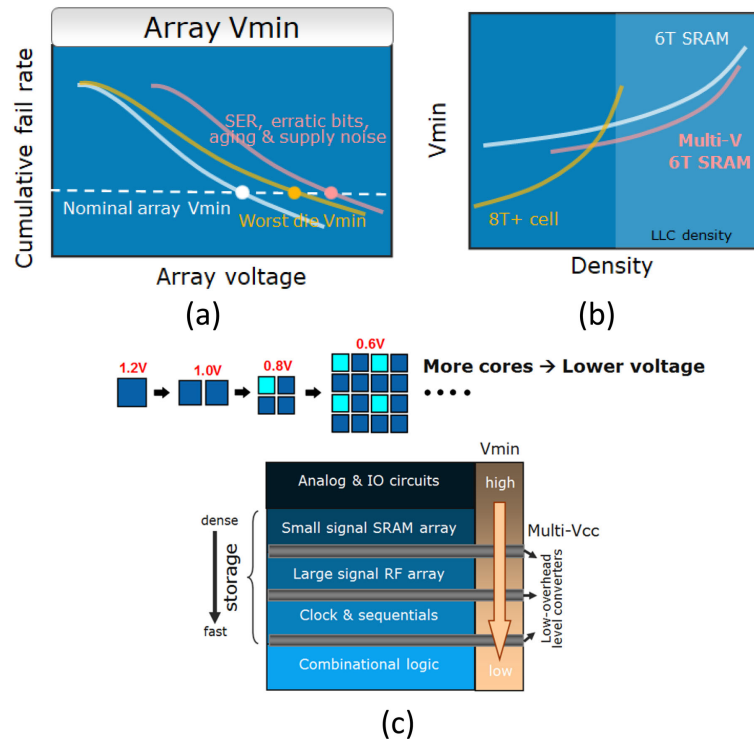


FIGURE 6. (a) Cache array Vmin. (b) Vmin versus density. (c) Multivoltage design.

voltage domains. More complex dynamic multivoltage read and write assist techniques that modulate individual transistor strengths during read and write operation are used to improve Vmin for cache arrays in 22-nm FinFET technology and beyond.^{10,11}

Cache leakage power dominates the total power when operating at low voltage and frequency, as well as under low activity and idle conditions. Therefore, fine-grain sleep transistor techniques are used to manage cache leakage in different power states and sleep modes. When the data need to be retained for faster wake-up, an active clamping circuit is used with the sleep transistor to ensure that the voltage does not fall below the data retention Vmin across process and temperature.^{12–14}

NEXT 25 YEARS IN CACHES

As described above, the last 25 years saw tremendous advances in caching, especially at the L3 cache, enabling distributed but shared caches, QoS awareness, guiding I/O data placement, enabling tighter coupling with accelerator memory and caching and more. Over the next 25 years, while difficult to predict, there are some major research challenges that will likely be important to address.

- 1) **Larger L4 caches and memory side caches:** It is clear that larger on-die cache capacity continues to be of importance as the workloads continue to grow in working sets (e.g., machine learning models growing rapidly). As a result, understanding the benefits and viability (cost) to build larger caches (integrated on-die, in cache tiles or three-dimensional stacked) is likely to be an area of focus. In addition, finding ways to reduce the on-die overhead of tag structures, snoop filters, or directories for larger caches also becomes critical to address. Memory disaggregation (expansion and pooling) at the system level is also an active exploration topic to improve the capacity and dynamic access of larger memory working sets and even better total cost of ownership (TCO) by pooling memory across multiple nodes. Using protocols like CXL.mem device-attached memory can be part of standard OS-managed system memory thus enabling disaggregated memory architectures. The remote memory controller receives the memory reads and writes from the CPU over CXL.mem and translates it into memory technology specific requests (e.g., requests to DDR memory connected to the

controller itself or transactions sent over to network-attached memory). The memory latency addition due to extending the memory hierarchy can be potentially mitigated by implementing memory side caching schemes that avoid the trip to the memory (locally or across a fabric). This memory side caching hierarchy may not be visible to the CPU, but it becomes important to understand when and how to cache hot pages in the memory side cache for effectiveness.

- 2) *New techniques for deeper cache hierarchies:* As cache hierarchies become deeper, the typical replacement policies start to become ineffective since recency information gets filtered through each level in the cache hierarchy. Finding better techniques to address this will become important to retain the efficiency of the caches. Deeper multilevel cache hierarchies also create challenges for system software to express locality/affinity domains, manage data persistence, contain errors when a failure occurs, create checkpoints for moving VM contexts, save and restore data while transitioning from different power states, and avoid performance interference and variability in the presence of diverse workloads sharing these resources. All of these areas will require continued research focus
- 3) *Compute in/near cache approaches* seem attractive to disrupt the traditional computing paradigm and research into the right primitives and techniques needed to make that mainstream would be very interesting. An example of compute in cache was recently explored to accelerate deep learning in the paper by Eckert *et al.*¹⁵ Fueled by such work, further research is well underway in the architecture community to find the most efficient approach while providing a balance between specialization and general-purpose use.

- 4) *Managing memory shared by accelerators and cores:* The use of scratchpads, buffers, and caches continue to remain independent between cores and accelerators. However, the growing presence of accelerators and the tight coupling of these with CPU cores have the potential to introduce new techniques for managing these caches as hybrid buffer/cache architectures. Furthermore, determining what to cache using hardware techniques versus what to expose for software management will continue to remain important as domain-specific processing (e.g., XPU for infrastructure processing and machine learning) emerges at large scale.
- 5) *Beyond SRAM for on-die caches* has been of interest for several years, but the mainstream use of these has remained constrained to specific platforms. In the next 25 years, with the ability to develop modular designs, heterogeneous caches and memories will emerge to enable different flows and data types to employ the right cache/memory technology for the right task. Methods to identify how to utilize these heterogeneous caches and memories will continue to be explored.
- 6) *AI-based techniques for caches* are being explored by researchers to further improve replacement policy decisions as well as other (QoS) placement and isolation decisions in the hierarchy. We expect this to help identify intelligent approaches especially as the workloads become more dynamic and efficient AI-based management of resources can provide better performance QoS.
- 7) *Higher level caching techniques at scale:* As large scale computing (i.e., warehouse scale) focuses on the use of microservices and RPCs to develop services and applications, higher level approaches like memoization may emerge for caching in future platforms. Such techniques require careful investigation since the interface for such caching becomes equally important as the identification of which aspects to cache and how to enable such caching (perhaps using AI techniques).

OVERALL, THE NOTION OF CACHE THAT WAS INTRODUCED BY MAURICE WILKES OVER 50 YEARS AGO HAS BECOME MAINSTREAM AND ADVANCED SIGNIFICANTLY WITH MULTICORE, MANYCORE, AND HETEROGENOUS PLATFORMS.

Overall, the notion of cache that was introduced by Maurice Wilkes over 50 years ago has become mainstream and advanced significantly with multicore, manycore, and heterogeneous platforms. Over the next 25 years, caches will continue to play a critical architectural role for enabling high performance, scalable computing platforms and enable richer capabilities (for higher density CPUs, XPU, memory disaggregation, and enabling large-scale systems) by 2050.

ACKNOWLEDGMENTS

We would like to thank A. Kumar for providing valuable feedback and input on this article.

REFERENCES

1. I. E. Papazian, "New 3rd Gen Intel Xeon scalable processor (Codename: Ice Lake-SP)," in *Proc. IEEE Hot Chips 32 Symp.*, 2020, pp. 1–22.
2. L. Zhao *et al.*, "NCID: A non-inclusive cache, inclusive directory architecture for flexible and efficient cache hierarchies," in *Proc. 7th ACM Int. Conf. Comput. Front.*, 2010, pp. 121–130.
3. A. Kumar *et al.*, "The new Intel Xeon scalable processor (formerly Skylake-SP)," in *Proc. Hot Chips 29*, Cupertino, CA, USA, Aug. 22, 2017. [Online]. Available: https://old.hotchips.org/wp-content/uploads/hc_archives/hc29/HC29.22-Tuesday-Pub/HC29.22.90-Server-Pub/HC29.22.930-Xeon-Skylake-sp-Kumar-Intel.pdf
4. R. Iyer, "CQoS: A framework for enabling QoS in shared caches of CMP platforms," in *Proc. Int. Conf. Supercomput.*, Saint-Malo, France, Jun. 2004, pp. 257–266.
5. R. Iyer *et al.*, "QoS policies and architecture for cache/memory in CMP platforms," in *Proc. Int. Conf. Meas. Model. Comput. Syst.*, San Diego, CA, USA, Jun. 2007, pp. 25–36.
6. A. Herdrich *et al.*, "Cache QoS: From concept to reality in the Intel Xeon E5-2600 v3 server processor family," in *Proc. Int. Conf. High Perform. Comput. Archit.*, 2016, pp. 657–668.
7. R. Huggahalli, R. Iyer, and S. Tetrack, "Direct cache access for high bandwidth network I/O," in *Proc. 32nd Int. Symp. Comput. Archit.*, Jun. 2005, pp. 50–59.
8. Y. Yuan *et al.*, "Don't forget the I/O when allocating your LLC," in *Proc. Int. Symp. Comput. Archit.*, 2021, pp. 112–125.
9. D. Sharma, "Compute express link," 2019. [Online]. Available: <https://www.computeexpresslink.org/post/introduction-to-compute-express-link-cxl-the-cpu-to-device-interconnect-breakthrough>
10. M. Khellah *et al.*, "PVT-Variations and supply-noise tolerant 45nm dense cache arrays with diffusion-notch-free (DNF) 6T SRAM cells and dynamic multi-Vcc circuits," in *Proc. Symp. Very Large Scale Integr. Circuits*, 2008, pp. 48–49.
11. E. Karl *et al.*, "A 4.6GHz 162Mb SRAM design in 22nm tri-gate CMOS technology with integrated active VMIN-enhancing assist circuitry," in *Proc. Int. Solid-State Circuits Conf.*, 2012, pp. 230–231.
12. M. Khellah *et al.*, "A 256-Kb dual-VCC SRAM building block in 65-nm CMOS process with actively clamped sleep transistor," *IEEE J. Solid-State Circuits*, vol. 42, no. 1, pp. 233–242, Jan. 2007.
13. R. Kumar and G. Hinton, "A family of 45nm IA processors," in *Proc. Int. Solid-State Circuits Conf.*, 2009, pp. 58–59.
14. N. Kurd *et al.*, "Westmere: A family of 32nm IA processors," in *Proc. Int. Solid-State Circuits Conf.*, 2010, pp. 96–97.
15. C. Eckert *et al.*, "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *Proc. Int. Symp. Comput. Archit.*, Los Angeles, CA, USA, Jun. 2018, pp. 383–396.
16. D. Mulnix, "Intel® Xeon® processor scalable-family technical overview," 2017. [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/intel-xeon-processor-scalable-family-technical-overview.html>
17. Intel® 64 and IA-32 Architectures Software Developer's Manual. [Online]. Available: <https://software.intel.com/content/www/us/en/develop/articles/intel-sdm.html>



RAVI IYER is an Intel Fellow and Director of the Emerging Systems Lab in Intel Labs, Hillsboro, OR, USA. He conducts research on emerging workloads, innovative architectures/technologies, and edge/cloud systems and infrastructure. He has authored/coauthored more than 150 papers and filed more than 75 patents. Iyer received a Ph.D. degree in computer science. He is a Fellow of IEEE. He is the corresponding author of this article. Contact him at ravishankar.iyer@intel.com.



VIVEK DE is an Intel Fellow and Director of Circuit Technology Research in Intel Labs, Hillsboro, OR, USA. He is responsible for providing strategic technical directions for long-term research in future circuit technologies and leading energy efficiency research across the hardware stack. He has 327 publications in refereed international conferences and journals with a citation H-index of 81, and 236 patents issued with 27 more patents filed (pending). He is a Fellow of IEEE. Contact him at vivek.de@intel.com.



RAMESH ILLIKKAL is a Principal Engineer with the Datacenter Product Group of Intel, Folsom, CA, USA. His research interests include datacenter QoS architecture and system benchmarking. He has authored/coauthored more than 75 papers and filed more than 50 patents. Contact him at Ramesh.g.illikkal@intel.com.



david.a.koufaty@intel.com.

DAVID KOUFATY is a Principal Engineer with Intel Labs, Hillsboro, OR, USA, where he focuses on systems architecture research for the next-generation scalable platform. He has more than 25 published research papers and 30 patents granted. Contact him at



bhushan.chitlur@intel.com.

BHUSHAN CHITLUR is a Senior Principal Engineer with the Programmable Solutions Group, Intel Labs, Hillsboro, OR, USA, focusing on next generation of datacenter accelerator and memory solutions using FPGAs and custom ASICs. He has more than 15 publications and 15 patents. Contact him at



at Andrew.J.Herdrich@intel.com.

ANDREW HERDRICH is a Principal Engineer with Intel Labs, Hillsboro, OR, USA, focusing on next-generation Xeon architecture, Intel RDT, and communications features. He holds 30 U.S. patents granted, and has authored/coauthored a number of research papers. Contact him at



muhammad.m.khellah@intel.com.

MUHAMMAD KHELLAH is a Principal Research Scientist with Intel Labs, Hillsboro, OR, USA, where he leads advanced research on power management, design technology co-optimization, and application-driven hardware acceleration. He has authored/coauthored more than 85 technical papers, and has more than 123 patents granted, and a few pending. Contact him at



fatih.hamzaoglu@intel.com.

FATIH HAMZAOGLU is an Intel Fellow. He joined Intel Technology Development in 2002. He leads new memory technology development, IP design, and product integration, such as eDRAM, MRAM, and RRAM. He has coauthored more than 40 papers and



eric.a.karl@intel.com.

ERIC KARL is an Intel Fellow and Director of Advanced Design in Intel's Technology Development organization, Hillsboro, OR, USA. He is responsible for design-technology co-optimization and IP development spanning standard cell logic libraries, embedded memories, high-speed DDR, SERDES I/O, phase-locked loops, and embedded programmable memory. He has authored/coauthored more than 30 papers in international conferences/journals and has been serving on the technical program committee for the IEEE International Solid-State Circuits Conference since 2019. Contact him at